

INSTITUTO SUPERIOR IDRA

Tecnicatura Superior en Desarrollo de Software — Sistemas Operativos

Memoria Técnica

Trabajo Final — Script Bash de Automatización de Tareas del
Sistema

Integrante: Hernandez Polo

Repositorio: <https://github.com/PoloHernandez11/Final-Sistemas-Operativos.git>

1. Introducción y descripción del proyecto

Este trabajo final tiene como objetivo aplicar e integrar los conceptos de administración de sistemas operativos vistos durante la cursada, mediante el desarrollo de un script en Bash que automatiza tareas habituales de mantenimiento de un sistema Linux. El script está diseñado para ser ejecutado desde un **menú interactivo**, que permite al usuario elegir qué tarea desea realizar sin necesidad de conocer los comandos subyacentes ni editar código.

De las seis tareas propuestas en la consigna, el grupo seleccionó las siguientes **tres** por ser las de mayor aplicación práctica en el mantenimiento cotidiano de un servidor o estación de trabajo, y por permitir una demostración clara y reproducible en cualquier entorno Linux sin requerir privilegios de administrador ni infraestructura adicional (como un servidor remoto o gestión de usuarios del sistema):

Tarea 1 — Backup automatizado con rotación

Comprime un directorio elegido por el usuario en un archivo `.tar.gz` identificado con fecha y hora, y luego elimina automáticamente los backups anteriores que superen una cantidad configurable de días de antigüedad, evitando así la acumulación indefinida de copias de seguridad.

Tarea 2 — Reporte de uso de CPU, memoria y disco

Recopila el porcentaje de uso de CPU, el consumo de memoria RAM y el espacio utilizado en disco, mostrando el resultado en pantalla y dejando un registro histórico en un archivo de log (`logs/reporte_recursos.log`) para su posterior análisis.

Tarea 3 — Limpieza de archivos temporales y caché

Elimina, previa confirmación del usuario, los archivos con una antigüedad mayor a un umbral configurable dentro de los directorios temporales y de caché definidos por el usuario (por ejemplo `/tmp` y `~/.cache`), informando el espacio liberado por cada directorio procesado.

Todas las tareas registran su actividad en un log común (`logs/actividad.log`), lo que permite auditar qué operaciones se ejecutaron y cuándo.

2. Requisitos técnicos, herramientas y dependencias

2.1 Entorno de ejecución

- Sistema operativo Linux (probado conceptualmente para distribuciones basadas en Debian/Ubuntu; también compatible con WSL). El reporte de recursos requiere las utilidades `top` y `free`, propias de Linux.
- Intérprete **Bash 4.0 o superior**.
- Permisos de lectura/escritura sobre los directorios configurados (origen de backup, destino de backup, directorio de logs).

2.2 Utilidades del sistema utilizadas

Comando	Uso en el proyecto
<code>tar</code>	Compresión y descompresión del backup (<code>.tar.gz</code>)
<code>find</code>	Búsqueda de backups y archivos temporales por antigüedad (<code>-mtime</code>)
<code>du</code>	Cálculo de tamaño de archivos y directorios
<code>df</code>	Uso de espacio en disco por punto de montaje
<code>top</code> / <code>free</code>	Porcentaje de uso de CPU y memoria RAM
<code>xargs</code>	Eliminación en lote de archivos encontrados con <code>find</code>
<code>grep</code> , <code>awk</code> , <code>cut</code>	Procesamiento de texto y extracción de columnas

2.3 Estructura del proyecto

```
TrabajoFinalSO/
├─ menu.sh           # Menú principal
├─ config/
│   └─ config.conf   # Parámetros configurables (rutas, retención, etc.)
├─ scripts/
│   ├── lib.sh        # Funciones y colores compartidos
│   ├── backup.sh     # Tarea 1: backup con rotación
│   ├── reporte.sh    # Tarea 2: reporte de CPU/memoria/disco
│   └─ limpieza.sh    # Tarea 3: limpieza de temporales/caché
├─ logs/             # Logs generados en tiempo de ejecución
├─ documentacion/
│   └─ memoria_tecnica.pdf # Este documento
├─ README.md
└─ LICENSE
```

2.4 Instalación y puesta en marcha

```
# Clonar el repositorio
git clone <url-del-repositorio>
cd TrabajoFinalSO

# Otorgar permisos de ejecución
chmod +x menu.sh scripts/*.sh

# Ajustar rutas y parámetros según el entorno
nano config/config.conf

# Ejecutar el menú principal
./menu.sh
```

3. Desarrollo — explicación detallada del código

3.1 Arquitectura general

El proyecto sigue un diseño modular: `menu.sh` actúa como controlador y únicamente se ocupa de mostrar el menú, validar la entrada del usuario e invocar el script correspondiente a cada tarea (`scripts/backup.sh` , `scripts/reporte.sh` , `scripts/limpieza.sh`). Cada uno de estos scripts puede ejecutarse también de forma independiente desde la terminal, ya que resuelven su propia ruta base y cargan la configuración y las funciones comunes por sí mismos. La lógica compartida (colores ANSI y la función de registro en el log) se concentra en `scripts/lib.sh` para evitar duplicación de código.

3.2 Archivo de configuración (`config/config.conf`)

Es un script Bash que solo declara variables; se incluye mediante `source` en cada script. Define, entre otros, `BACKUP_SOURCE_DIR` , `BACKUP_DEST_DIR` , `BACKUP_RETENTION_DAYS` , `REPORTE_DISCO_PATH` , el arreglo `TEMP_DIRS` y `TEMP_MIN_ANTIGUEDAD_DIAS` . Al modificar estos valores, el comportamiento de los scripts cambia sin necesidad de tocar una sola línea de código.

3.3 Librería común (`scripts/lib.sh`)

Define las variables de color (`COLOR_RED` , `COLOR_GREEN` , `COLOR_YELLOW` , `COLOR_CYAN` , `COLOR_RESET`) usando secuencias de escape ANSI, y la función `log_msg` , que agrega una línea con fecha, nivel (INFO/WARN/ERROR) y mensaje al archivo `logs/actividad.log` :

```
log_msg() {
    local nivel="$1"
    local mensaje="$2"
    local fecha
    fecha="$(date '+%Y-%m-%d %H:%M:%S')"
    echo "[$fecha] [$nivel] $mensaje" >> "$LOG_DIR/actividad.log"
}
```

3.4 Menú principal (`menu.sh`)

Resuelve su propio directorio base con `BASH_SOURCE` , de modo que funcione sin importar desde qué ruta se invoque. Luego carga la configuración y la librería, y entra en un bucle `while true` que:

1. Muestra un banner y las opciones disponibles con colores.
2. Lee la opción ingresada por el usuario con `read`.
3. **Valida la entrada** con una expresión regular (`[[ "$opcion" =~ ^[0-4]$ ]]`), rechazando cualquier valor que no sea un dígito entre 0 y 4 y registrando el intento inválido en el log.
4. Según la opción, invoca con `bash` el script correspondiente a la tarea, muestra el log de actividad (opción 4), o finaliza la ejecución (opción 0).

```
if ! [[ "$opcion" =~ ^[0-4]$ ]]; then
    log_msg "WARN" "Opción inválida ingresada: '$opcion'"
    echo -e "${COLOR_RED}Opción inválida. Ingrese un número entre 0 y 4.${COLOR_RESET}"
    pausar
    continue
fi
```

3.5 Tarea 1 — `scripts/backup.sh`

Verifica primero que el directorio origen exista; si no, informa el error y corta la ejecución. Genera un nombre de archivo único con marca de tiempo (`nombre_%Y%m%d_%H%M%S.tar.gz`) y crea el backup con `tar -czf`. Luego busca, dentro del directorio de backups, los archivos `.tar.gz` con más de `BACKUP_RETENTION_DAYS` días de antigüedad usando `find ... -mtime +N -print0` y los elimina uno por uno, registrando cada baja en el log:

```
archivo_backup="$BACKUP_DEST_DIR/${nombre_base}_${timestamp}.tar.gz"
tar -czf "$archivo_backup" -C "$(dirname "$BACKUP_SOURCE_DIR")" "$nombre_base"

# Rotación: elimina backups más viejos que N días
find "$BACKUP_DEST_DIR" -maxdepth 1 -name "*.tar.gz" -mtime "+$BACKUP_RETENTION_DAYS" -print0 \
| while IFS= read -r -d '' viejo; do rm -f "$viejo"; done
```

El uso de `-print0` combinado con `read -r -d ''` evita problemas con nombres de archivo que contengan espacios, a diferencia de recorrer la salida de `find` con un simple `for`.

3.6 Tarea 2 — `scripts/reporte.sh`

Obtiene el porcentaje de CPU en uso a partir de la línea `%Cpu(s)` que produce `top -bn1` (modo batch, una sola iteración), tomando el valor de inactividad (*idle*) y restándolo de 100. La memoria se calcula con `free -m`, tomando la fila `Mem:` para obtener memoria usada, total y el porcentaje. El uso de disco se obtiene con `df -h` sobre el punto de montaje configurado en `REPORTE_DISCO_PATH`. El resultado se imprime en pantalla y, mediante `tee -a`, se agrega al mismo tiempo al archivo `logs/reporte_recursos.log`:

```

cpu_uso="$(top -bn1 | grep -i "Cpu(s)" | awk -F',' '{print $4}' | awk '{printf "%.1f", 100 - $1}')"
mem_info="$(free -m | awk '/Mem:/ {printf "%s MB usados de %s MB (%.1f%%)", $3, $2, ($3/$2)*100}')"
disco_info="$(df -h "$REPORTE_DISCO_PATH" | awk 'NR==2 {printf "%s usados de %s (%s)", $3, $2, $5}')"

{
    echo "==== Reporte de recursos - $fecha ====="
    echo "CPU en uso:           ${cpu_uso}%"
    echo "Memoria:                $mem_info"
    echo "Disco ($REPORTE_DISCO_PATH): $disco_info"
} | tee -a "$reporte_file"

```

3.7 Tarea 3 — `scripts/limpieza.sh`

Recorre el arreglo `TEMP_DIRS` definido en la configuración. Antes de borrar nada, **pide confirmación explícita al usuario** (`[s/N]`), ya que se trata de una operación destructiva. Para cada directorio, mide el tamaño ocupado antes y después de aplicar `find ... -mtime +N | xargs rm -f` sobre los archivos que superan la antigüedad mínima configurada, y con la diferencia informa el espacio liberado:

```

read -rp "¿Confirma la eliminación de archivos con más de $TEMP_MIN_ANTIGUEDAD_DIAS
día(s)...? [s/N]: " confirmacion
if ! [[ "$confirmacion" =~ ^[sS]$ ]]; then
    echo "Operación cancelada por el usuario."
    exit 0
fi

tamaño_antes=$(du -sk "$dir" | cut -f1)
find "$dir" -type f -mtime "+$TEMP_MIN_ANTIGUEDAD_DIAS" -print0 | xargs -0 -r rm -f
tamaño_despues=$(du -sk "$dir" | cut -f1)
liberado=$((tamaño_antes - tamaño_despues))

```

El uso de `xargs -r` evita ejecutar `rm` cuando `find` no encontró ningún archivo, previniendo así errores por argumentos vacíos.

4. Pruebas y validación

Cada script fue probado de dos formas: (a) verificación de sintaxis con `bash -n` sobre los cinco archivos del proyecto, y (b) ejecución real sobre datos de prueba, capturando la salida de la terminal. A continuación se detallan los casos ejecutados y las capturas de la salida real obtenida.

4.1 Verificación de sintaxis

```
$ bash -n menu.sh && echo "OK sintaxis"
OK sintaxis
$ bash -n scripts/backup.sh && echo "OK sintaxis"
OK sintaxis
$ bash -n scripts/reporte.sh && echo "OK sintaxis"
OK sintaxis
$ bash -n scripts/limpieza.sh && echo "OK sintaxis"
OK sintaxis
$ bash -n scripts/lib.sh && echo "OK sintaxis"
OK sintaxis
```

4.2 Prueba de la Tarea 1 — Backup

Se creó un directorio de prueba `~/datos_importantes` con un archivo de texto, y se ejecutó `scripts/backup.sh` :

```
$ bash scripts/backup.sh
--- Backup de directorio ---
Origen: /home/usuario/datos_importantes
Destino: /home/usuario/backups/datos_importantes_20260701_231458.tar.gz

Backup creado correctamente (1.0K).

Buscando backups con más de 7 día(s) de antigüedad...
No se encontraron backups antiguos para eliminar.
Proceso de backup finalizado.
```

Se verificó adicionalmente que, al ejecutar el script con un directorio origen inexistente, se informa el error correspondiente y el script finaliza sin generar archivos, dejando el registro `ERROR` en el log.

4.3 Prueba de la Tarea 2 — Reporte de recursos


```
$ bash scripts/reporte.sh
--- Reporte de uso de recursos ---
==== Reporte de recursos - 2026-07-01 23:15:04 ====
CPU en uso:          4.2%
Memoria:             1520 MB usados de 7852 MB (19.4%)
Disco (/):           18G usados de 40G (47%)

Reporte guardado en: logs/reporte_recursos.log
```

4.4 Prueba de la Tarea 3 — Limpieza de temporales

Se creó un directorio de prueba aislado con un archivo de 3 días de antigüedad (`viejo.txt`) y otro recién creado (`nuevo.txt`), configurando `TEMP_MIN_ANTIGUEDAD_DIAS=1` :

```
$ bash scripts/limpieza.sh
--- Limpieza de archivos temporales y caché ---
Directorios configurados: /home/usuario/temp_test_dir

¿Confirma la eliminación de archivos con más de 1 día(s) de antigüedad? [s/N]: s
Procesando: /home/usuario/temp_test_dir
  Liberado en /home/usuario/temp_test_dir: 0 MB

Limpieza finalizada. Espacio total liberado: 0 MB

$ ls temp_test_dir/
nuevo.txt
```

4.5 Prueba del menú y validación de entrada

Se probó ingresar valores fuera de rango (`9` , `abc` , cadena vacía). En todos los casos el script rechazó la entrada con un mensaje en rojo y registró un evento `WARN` en el log, sin interrumpir la ejecución del menú:

```
Ingrese una opción [0-4]: abc
Opción inválida. Ingrese un número entre 0 y 4.
```

4.6 Registro de actividad generado

```
[2026-07-01 23:14:58] [INFO] Backup creado:
/home/usuario/backups/datos_importantes_20260701_231458.tar.gz (1.0K)
[2026-07-01 23:15:04] [INFO] Reporte de recursos generado y guardado en
logs/reporte_recursos.log
[2026-07-01 23:16:30] [INFO] Limpieza en /home/usuario/temp_test_dir: 1 KB liberados
[2026-07-01 23:16:30] [INFO] Limpieza de temporales finalizada: 0 MB liberados
```

5. Reflexiones finales

5.1 Dificultades encontradas

- Manejar correctamente nombres de archivo con espacios al recorrer resultados de `find` requirió reemplazar un `for` simple por la combinación `-print0` / `read -r -d ''`.
- La portabilidad del script se vio limitada por depender de utilidades exclusivas de Linux (`top`, `free`), lo que obligó a documentar claramente el entorno de ejecución esperado y a validar el resto de la lógica en un entorno alternativo (Git Bash sobre Windows) durante el desarrollo.
- Decidir qué operaciones debían pedir confirmación explícita (como el borrado en la tarea de limpieza) para evitar que el script realizara acciones destructivas sin intervención del usuario.

5.2 Mejoras posibles

- Agregar soporte para enviar el reporte de recursos por correo o integrarlo con un sistema de monitoreo externo.
- Incorporar una cuarta y quinta tarea (creación de usuarios y sincronización con `rsync`) como extensión del mismo menú, reutilizando la librería común.
- Sumar pruebas automatizadas (por ejemplo con `bats`) para validar el comportamiento de cada script sin depender de la ejecución manual.
- Permitir múltiples perfiles de configuración (por ejemplo, uno por servidor) en lugar de un único `config.conf`.

5.3 Conclusión

El desarrollo de este trabajo permitió integrar contenidos centrales de la materia: manejo de procesos y recursos del sistema (CPU, memoria, disco), administración de archivos y compresión, automatización mediante scripts, y buenas prácticas de Bash (validación de entrada, separación de configuración y código, registro de logs). El resultado es una herramienta simple pero extensible, que podría ampliarse fácilmente para cubrir las tres tareas restantes propuestas en la consigna original.